

Deep Learning – Homework 1

Introduction, Optimization, and Convolutional Neural Networks

Framework: PyTorch | Datasets: MNIST (Problems 1–2), CIFAR-10 (Problem 3) | Seed: 42

Problem 1: Implementing and Training MLPs

Part A — MLP Implementation

A flexible MLP class was implemented supporting an arbitrary number of hidden layers, selectable activation functions (ReLU, Sigmoid, Tanh), and a softmax output. The model was trained on MNIST with the required $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ architecture using the Adam optimizer for 10 epochs.

Metric	Value
Final training accuracy	99.23%
Final test accuracy	97.67%
Training time	26.3 s

Part B — Activation Function Comparison

The same $784 \rightarrow 128 \rightarrow 64 \rightarrow 10$ architecture was trained for 10 epochs with three activation functions (ReLU, Sigmoid, Tanh), keeping all other settings identical for a fair comparison.

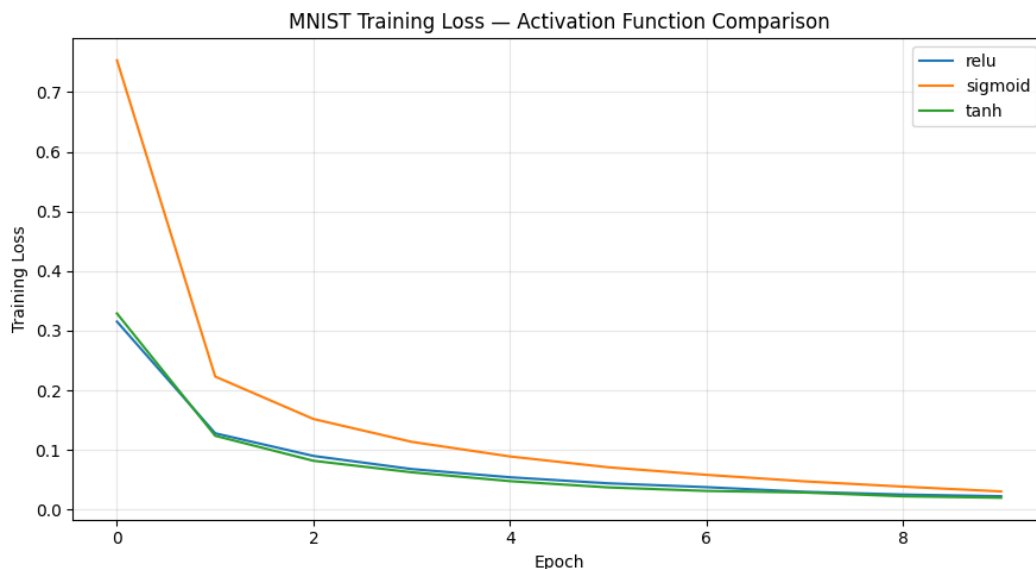


Figure 1. Training loss curves for ReLU, Sigmoid, and Tanh on MNIST.

Activation	Final Train Acc.	Final Test Acc.	Training Time (s)
ReLU	99.23%	97.67%	25.26
Sigmoid	99.52%	97.58%	25.44

Tanh	99.57%	97.77%	25.69
------	--------	--------	-------

Analysis: All three activations converge to comparable test accuracy (97.6–97.8%) on this shallow network, with Tanh achieving the highest final test accuracy (97.77%) and the lowest final training loss, closely followed by ReLU. Sigmoid converges the slowest in the early epochs (loss of 0.75 after epoch 1 versus about 0.32 for ReLU and Tanh) because its outputs and gradients saturate for inputs far from zero, which slows early learning. ReLU and Tanh avoid this problem for typical input magnitudes and therefore optimize faster from the start. On this small, shallow network the vanishing-gradient effect of sigmoid is mild, so the accuracy gap between the three activations remains small; the effect becomes far more pronounced for deeper networks, as shown in Problem 2B.

Part C — Solving XOR

A small MLP (2 inputs → 4 hidden neurons, Tanh → 1 sigmoid output) was trained on the four XOR cases.

x1	x2	Target	Predicted Prob.	Prediction	Correct
0	0	0	0.0000	0	Yes
0	1	1	0.9999	1	Yes
1	0	1	0.9999	1	Yes
1	1	0	0.0002	0	Yes

Final accuracy on all four XOR cases: **100%** (4/4).

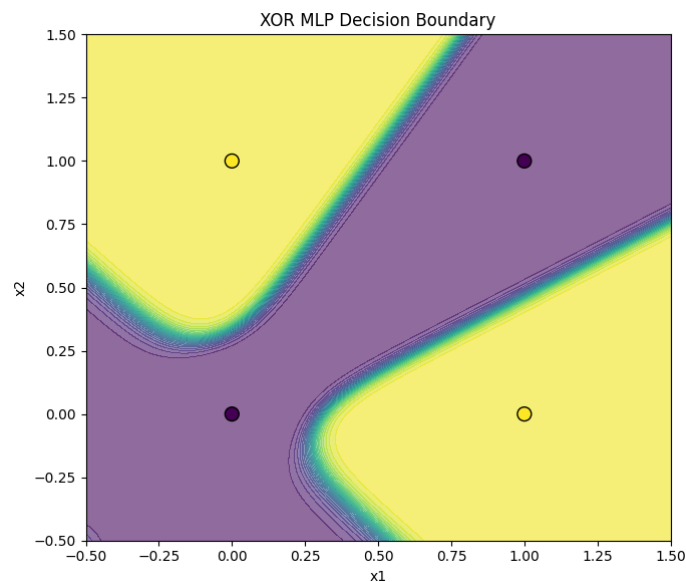


Figure 2. Decision boundary learned by the XOR MLP; markers show the four training points.

Problem 2: Optimization and Training Dynamics

Part A — Optimizer Comparison

The Problem 1A architecture was trained for 20 epochs on MNIST using four optimizers: SGD ($\text{lr}=0.01$), SGD with momentum (momentum=0.9, $\text{lr}=0.01$), RMSprop ($\text{lr}=0.001$), and Adam ($\text{lr}=0.001$), all from an identical initialization.

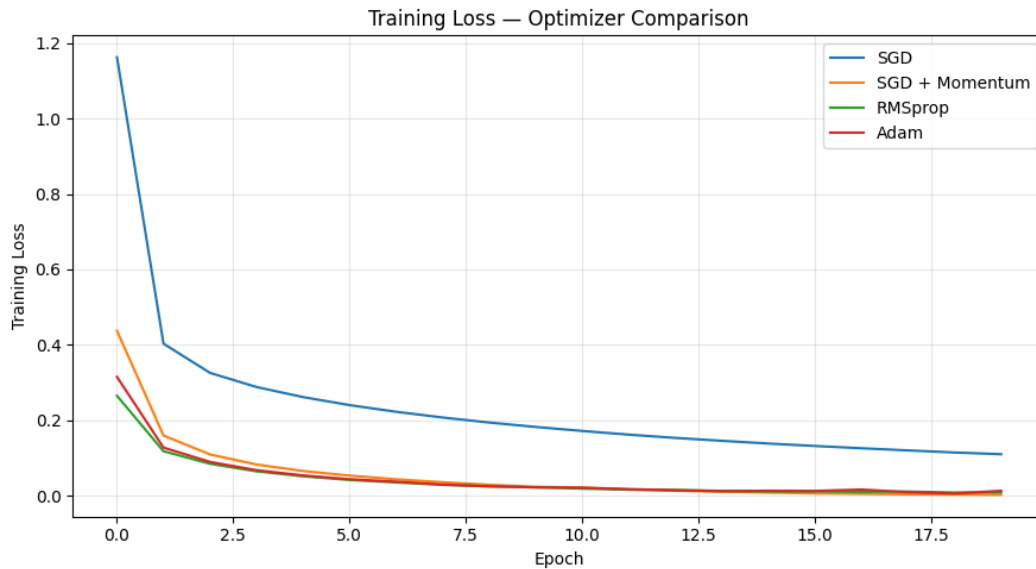


Figure 3. Training loss vs. epoch for each optimizer.

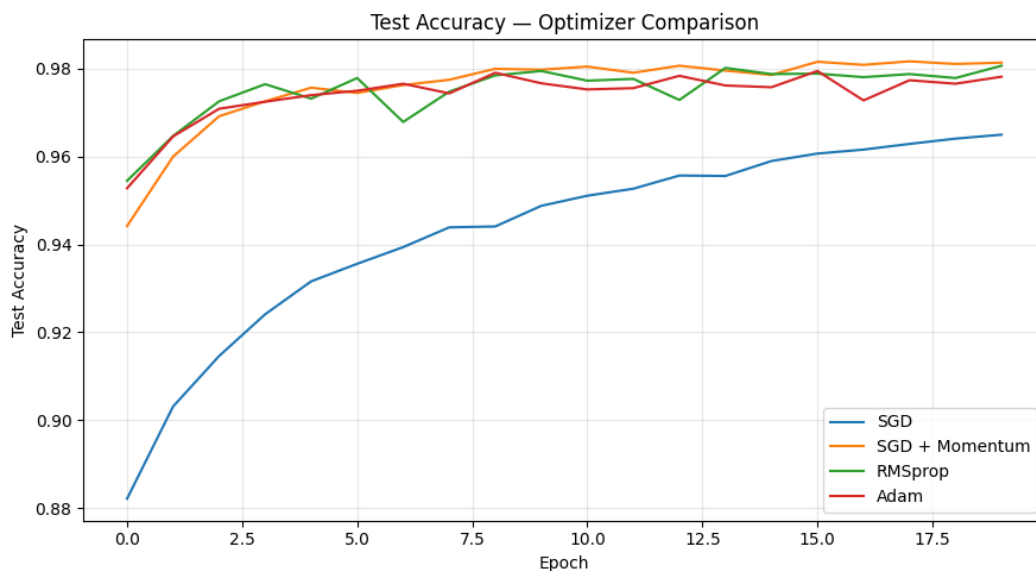


Figure 4. Test accuracy vs. epoch for each optimizer.

Optimizer	Final Test Acc.	Training Time (s)
SGD	96.50%	51.58
SGD + Momentum	98.14%	51.11
RMSprop	98.07%	51.47

Adam	97.82%	50.91
------	--------	-------

Analysis: Plain SGD converges markedly slower than the other three methods, reaching only 88.2% test accuracy after epoch 1 and finishing at 96.50% after 20 epochs, since it applies the same fixed step to every parameter with no memory of past gradients. Adding momentum dramatically accelerates convergence (94.4% after epoch 1, 98.14% final, the best of all four) because it accumulates velocity along consistent gradient directions and damps oscillations. RMSprop, which adapts the learning rate per parameter using a running average of squared gradients, converges almost as fast and finishes at 98.07%. Adam, which combines momentum with per-parameter adaptive rates, converges quickly but finishes slightly behind SGD+Momentum and RMSprop here (97.82%), which can happen because its adaptive scaling sometimes generalizes slightly worse than well-tuned momentum-based SGD on simple tasks. All four optimizers take essentially the same wall-clock time per epoch, since runtime is dominated by the forward/backward passes rather than the optimizer update rule.

Part B — Gradient Analysis

A deep MLP with 8 hidden layers of 128 neurons was used to study the vanishing-gradient problem. Mean absolute gradient magnitudes were tracked across layers for Sigmoid and ReLU activations.

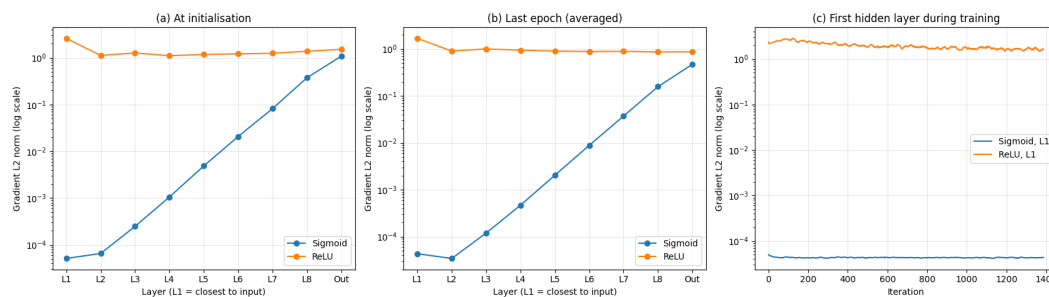


Figure 5. Gradient magnitude by layer depth for Sigmoid vs. ReLU activations.

Quantity	Sigmoid	ReLU
Grad(Layer 1) / Grad(Layer 8) at init.	1.37e-04	1.88e+00
Final training loss	2.30	0.159

Explanation: With Sigmoid activations the gradient at layer 1 is roughly four orders of magnitude smaller than at layer 8 (ratio $\approx 1.37\text{e-}04$), confirming the vanishing-gradient effect: each layer's sigmoid derivative is at most 0.25, so gradients shrink multiplicatively as they are backpropagated through eight layers, leaving the early layers with almost no learning signal and a high final training loss (2.30). With ReLU, whose derivative is exactly 1 for all positive pre-activations, gradients are preserved (and can even grow slightly) across depth, giving a layer-1/layer-8 gradient ratio near 1 (1.88e+00) and a much lower final training loss (0.159). This directly illustrates why ReLU-family activations are preferred in deep networks trained with plain gradient-based methods.

Part C — Regularization Effects

Three configurations of the Problem 1A architecture were trained for 10 epochs: a baseline with no regularization, L2 weight decay of 0.001, and dropout with $p=0.5$ on the hidden layers.

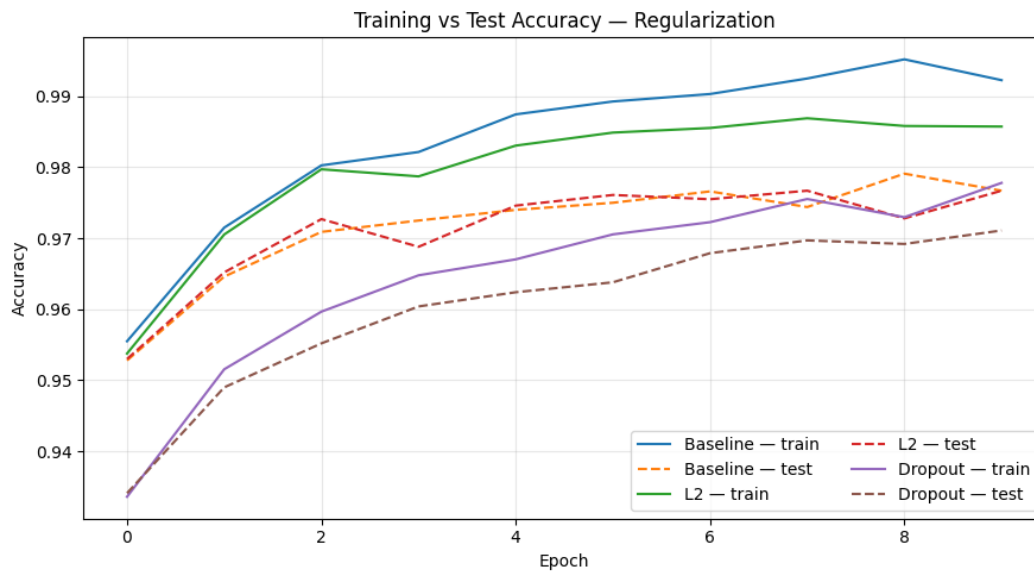


Figure 6. Training vs. test accuracy for baseline, L2, and dropout configurations.

Configuration	Train Acc.	Test Acc.	Train-Test Gap
Baseline	99.23%	97.67%	1.56%
L2 (weight decay = 0.001)	98.57%	97.67%	0.90%
Dropout (p = 0.5)	97.78%	97.11%	0.67%

Analysis: The baseline model shows the largest train–test gap (1.56 points), indicating mild overfitting by epoch 10. Dropout produces the smallest gap (0.67 points) by randomly deactivating hidden units during training, which prevents co-adaptation of features, though it also lowers both training and test accuracy since the network is effectively regularized more aggressively. L2 regularization narrows the gap to 0.90 points while keeping test accuracy identical to the baseline (97.67%), making it the best generalization-per-accuracy trade-off in this experiment, while dropout is the most effective at directly shrinking the generalization gap.

Problem 3: Convolutional Neural Networks

Part A — Basic CNN Implementation

The required CNN (Conv32–Pool–Conv64–Pool–FC128–FC10, 3×3 kernels, same padding) was trained for 20 epochs on CIFAR-10 (45,000 train / 5,000 validation / 10,000 test images).

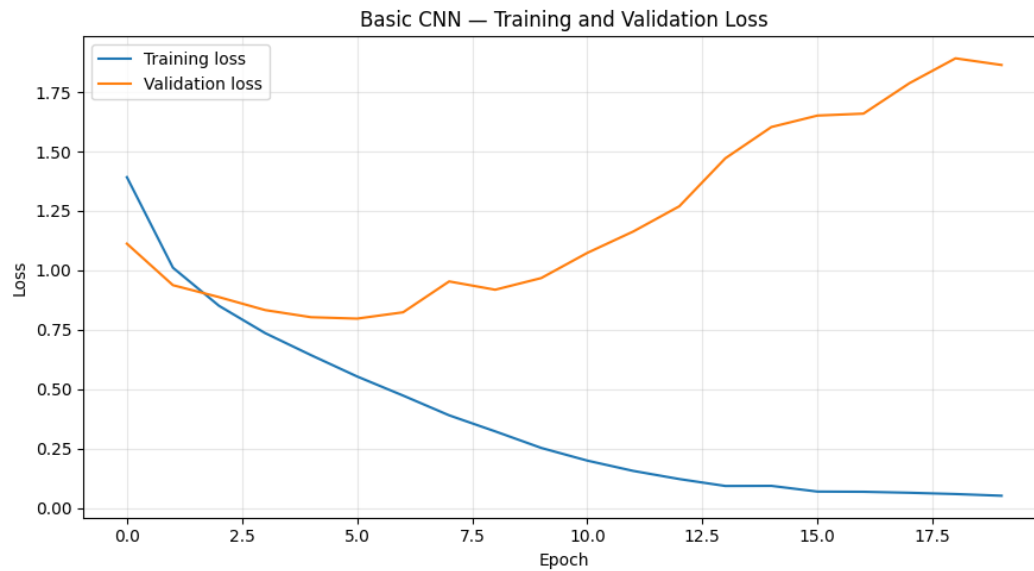


Figure 7. Basic CNN training and validation loss curves.

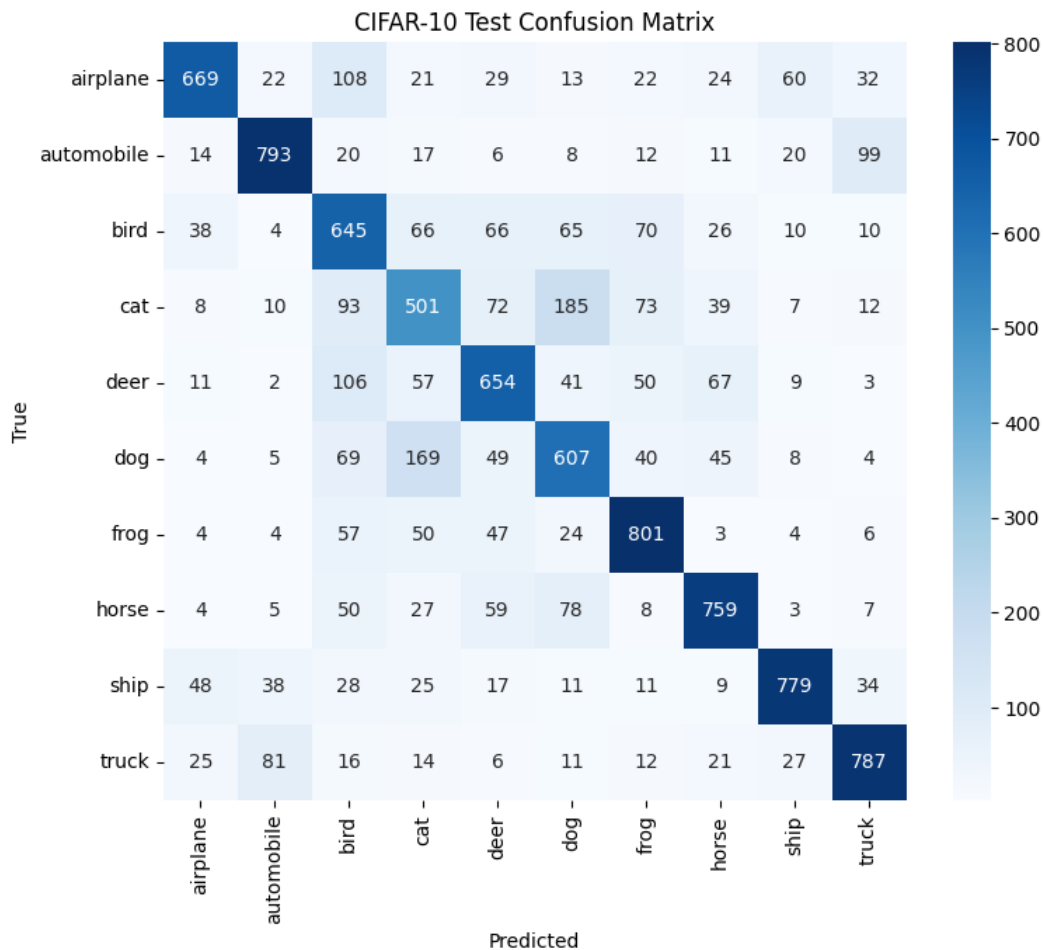


Figure 8. Confusion matrix on the CIFAR-10 test set.

Metric	Value
Final test accuracy	69.95%
Training time (20 epochs)	585.6 s
Best validation loss	0.797 (epoch 6)

The validation loss reaches its minimum at epoch 6 (0.797) and then rises steadily to 1.86 by epoch 20 while training loss keeps falling, indicating that the model overfits the training set well before training ends.

Part B — Architecture Experimentation

Three additional CNN architectures were designed and compared against the Part A baseline, varying depth, filter counts, kernel size, and pooling strategy.

Model	Conv Layers	Filters	Kernels	Pooling	Idea
Baseline (3A)	2	32 / 64	3×3	Max	Required architecture, reference
Architecture A	2	16 / 32	5×5	Max	Smaller & shallower: fewer params
Architecture B	3	32 / 64 / 128	3×3	Max	Deeper & wider: more capacity

Architecture C	3	32 / 64 / 64	5×5, 5×5, 3×3	Average	Average instead of max pooling
----------------	---	--------------	---------------	---------	--------------------------------

Model	Parameters	Training Time (s)	Test Accuracy
Baseline (3A)	545,098	585.6	69.95%
Architecture A	277,610	572.6	67.55%
Architecture B	1,143,242	1385.7	73.02%
Architecture C	616,330	842.3	70.48%

Trade-off analysis: Architecture B (deeper, wider, more filters) achieves the highest test accuracy (73.02%) but at roughly 2.1× the parameters and 2.4× the training time of the baseline, showing that added capacity does translate into better accuracy here, at a steep computational cost. Architecture A, which reduces filter counts to save parameters, trains fastest but achieves the lowest accuracy (67.55%), confirming that under-capacity limits what the network can learn. Architecture C, which swaps max pooling for average pooling and uses larger early kernels, lands between the baseline and Architecture B in both accuracy (70.48%) and cost, suggesting max pooling is mildly preferable to average pooling for this task since it preserves the strongest activations rather than smoothing them out. Overall, model size and accuracy trade off roughly monotonically in this comparison, so the best choice depends on whether the deployment prioritizes accuracy or training/inference cost.

Part C — Visualizing Learned Features

The learned first-layer convolutional filters and their resulting activation maps were visualized for the Part A CNN.

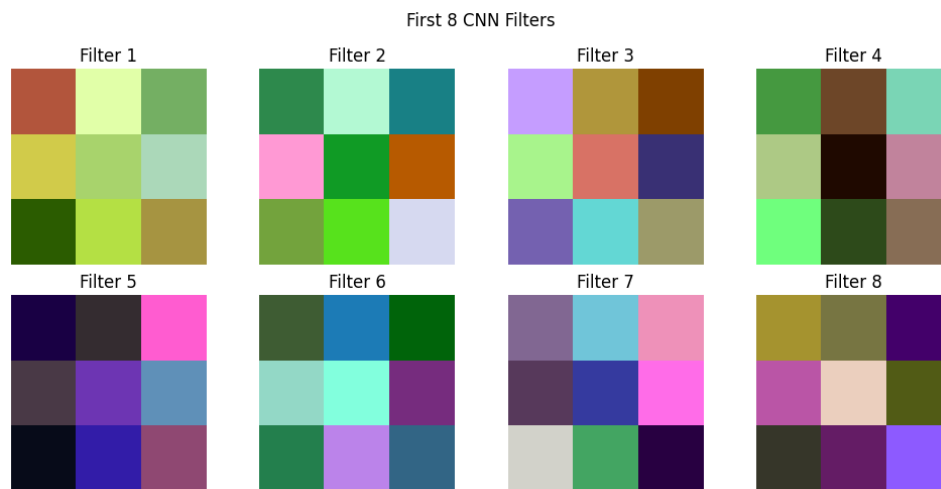


Figure 9. First 8 filters (kernels) of the first convolutional layer.

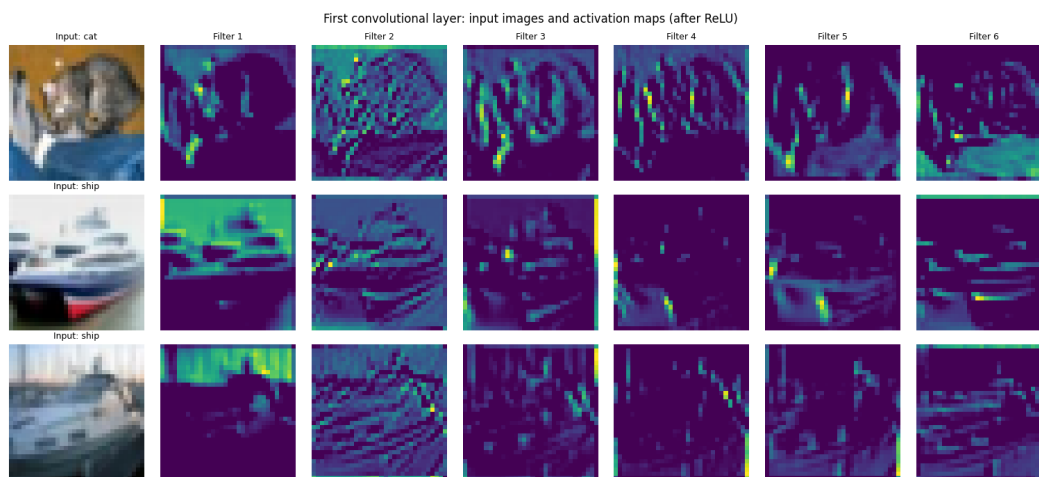


Figure 10. Input images (left column) and their first-layer activation maps (after ReLU) for three CIFAR-10 test samples.

The first-layer filters capture low-level, generic image features such as color-opponent blobs, edges, and simple color/texture gradients — typical of the earliest layer in a CNN. The corresponding activation maps show that different filters respond selectively to different regions and structures of each input image (e.g., object boundaries or contrast regions), which is consistent with the filters acting as basic edge/color detectors that feed richer, more abstract representations in later layers.

Bonus: Transfer Learning (Not Attempted)

The optional transfer-learning bonus (fine-tuning a pre-trained ResNet18/VGG16 on CIFAR-10) was not implemented in this submission; Problem 3A's CNN trained from scratch (69.95% test accuracy) serves as the comparison baseline should this extension be attempted.

End of report.